Implementation plan: tenancy and IAM (v4.1)

Builds: the "User, access control and tenancy v4" tab · **Stack:** PHP 8.4 FPM, Symfony 8.1, API Platform 5, Doctrine ORM 3 / DBAL 4, PostgreSQL 17, Redis, Nginx + Certbot (production), Symfony CLI server (local) · **No Docker, no FrankenPHP** · **Order:** phases are sequential; steps inside a phase can be split between people where marked.

Each step says what to create, the commands to run, and how to prove it works. Tests are Pest. Nothing is marked done until its test passes.

Phase 0: workstation and repository

0.1 Local services

Already installed with Homebrew: `php@8.4`, `postgresql@17`, `redis`, `nginx`. Also install:

```
brew install symfony-cli composer
brew services start postgresql@17 redis
psql postgres -c '\du'   # confirms your macOS user is the superuser
```

Nginx stays stopped locally; the Symfony CLI server serves `*.localhost` with a local CA.

0.2 Dependencies to approve and install

Package	Why
`league/oauth2-server-bundle`	Authorization server (code + PKCE, client credentials, refresh rotation)
`web-token/jwt-framework`	ES256 JWS, JWKS, `kid`
`web-auth/webauthn-symfony-bundle`	Passkeys
`symfony/workflow`	Onboarding state machine
`symfony/scheduler`, `symfony/messenger`, `symfony/lock`, `symfony/cache`	Pool filler, sagas, locks, Redis cache
`jeremykendall/php-domain-parser`	Public Suffix List check for custom domains
`api-platform/test` (dev)	`ApiTestCase`

Remove: `lexik/jwt-authentication-bundle`, `gesdinet/jwt-refresh-token-bundle`.

0.3 Environment

`.env.local`:

```
APP_ENV=dev
TENANCY_BASE_DOMAIN=localhost
LANDLORD_DATABASE_URL=postgresql://colsynq_app:dev@127.0.0.1:5432/colsynq_landlord?serverVersion=17
TENANT_DATABASE_URL=postgresql://colsynq_app:dev@127.0.0.1:5432/placeholder?serverVersion=17
TENANT_MIGRATION_URL=postgresql://colsynq_migrator:dev@127.0.0.1:5432/placeholder?serverVersion=17
MESSENGER_TRANSPORT_DSN=doctrine://landlord
REDIS_URL=redis://127.0.0.1:6379
SECRETS_PROVIDER=env
```

0.4 Directory layout

```
src/
  Shared/Tenancy/        TenantId, ResolvedTenant, TenantContext,
TenantResolver, TenantScopedRunner,
                         TenantDirectory (port), TenantConnectionResolver
(port), TenantRequirement, exceptions
  Shared/Identity/       CurrentActor (port), ActorId, ProfileRef
  Shared/Messaging/      Outbox, Inbox, TenantStamp, LandlordMessage, event
base classes
  Shared/Audit/          AuditWriter
  Tenancy/               Domain, Application, Infrastructure (Doctrine, Api,
Console, Messenger, Scheduler)
  Iam/                   Domain, Application, Infrastructure (Doctrine, Api,
Security, OAuth, Console)
config/packages/tenancy.yaml, doctrine.yaml, messenger.yaml, workflow.yaml,
security.yaml, api_platform.yaml
migrations/landlord/, migrations/tenant/
tests/Architecture/, tests/Tenancy/, tests/Iam/, tests/Isolation/
```

Architecture rule from day one (Pest `arch()` test): `Tenancy` and `Iam` may depend on `Shared` and on vendors only; business modules may depend on `Shared` only; nothing depends on `Tenancy` or `Iam` directly.

Phase 1: database foundation

1.1 `tenancy:setup-cluster` command

Turns a bare PostgreSQL server into a cluster. Runs once per server with a superuser URL, and again at any time to verify nothing drifted; every step checks before it acts, so re-running converges instead of failing. It is the only place superuser access is ever used.

```
bin/console tenancy:setup-cluster local \
  --admin-url=postgresql://$USER@127.0.0.1:5432/postgres \
  --with-landlord
```

Options: `<cluster>` (must exist in `tenancy.yaml`; the admin URL host must match its direct endpoint), `--admin-url`, `--with-landlord` (first cluster only), `--reset-passwords` (otherwise existing roles keep their passwords), `--check` (verification only, no changes). Passwords come from `COLSYNQ_APP_PASSWORD`, `COLSYNQ_MIGRATOR_PASSWORD`, `PGBOUNCER_AUTH_PASSWORD` and must match the connection URLs in the environment.

Step	Statements (each guarded by an existence check)	Why
1 Roles	`CREATE ROLE colsynq_migrator LOGIN CREATEDB NOSUPERUSER NOCREATEROLE NOINHERIT`; `CREATE ROLE colsynq_app LOGIN NOSUPERUSER NOCREATEROLE NOCREATEDB`; `CREATE ROLE pgbouncer LOGIN NOINHERIT`	Migrator owns and creates; app only reads and writes rows; neither can mint logins or escalate
2 Landlord (`--with-landlord`)	`CREATE DATABASE colsynq_landlord OWNER colsynq_migrator ENCODING 'UTF8' TEMPLATE template0`; in it: `REVOKE ALL ON DATABASE … FROM PUBLIC`, `GRANT CONNECT … TO colsynq_app`, `REVOKE CREATE ON SCHEMA public FROM PUBLIC`	`template0` is pristine; schemas come from the landlord migration, not from here

Step	Statements (each guarded by an existence check)	Why
3 Template	`CREATE DATABASE tenant_template OWNER colsynq_migrator ENCODING 'UTF8' TEMPLATE template0`; `ALTER DATABASE tenant_template IS_TEMPLATE = true`; in it: the same REVOKE and GRANT CONNECT, `GRANT USAGE ON SCHEMA public TO colsynq_app`, and `ALTER DEFAULT PRIVILEGES FOR ROLE colsynq_migrator IN SCHEMA public GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES / USAGE, SELECT ON SEQUENCES / EXECUTE ON FUNCTIONS TO colsynq_app`	Default privileges are rules copied with the template, so every table a migration creates in every pool database is usable by the app without GRANT statements in migrations. Business schemas repeat the rule in their first migration
4 PgBouncer auth	`CREATE DATABASE pgbouncer_auth OWNER colsynq_migrator TEMPLATE template0`; in it a `SECURITY DEFINER` function `lookup_user(text)` returning `usename, passwd` from `pg_shadow` only for `colsynq_app` and `colsynq_migrator`; EXECUTE granted to `pgbouncer` only	`auth_query` cannot run inside tenant databases (they refuse PUBLIC), and the function can never leak another role's hash. Created locally too, so every server is identical
5 Verify	As `colsynq_migrator`: create and drop a probe database from the template (succeeds). As `colsynq_app`: connect to `tenant_template` (succeeds); `CREATE TABLE` there (denied); connect to `postgres` (denied). As `pgbouncer`: `lookup_user('colsynq_app')` returns one row, `lookup_user('postgres')` returns none	A misconfigured server fails here, before it can hold tenant data

Step	Statements (each guarded by an existence check)	Why
6 Record	`audit.event` row (`action = cluster.setup`) when the landlord schema exists; otherwise a printed summary	First run has no audit table yet

Not done here, on purpose: migrations (phase 1.3 and 1.5, as the migrator), tenant databases (pool filler at run time), storing passwords (environment or secrets manager only), `pg_hba.conf` and `postgresql.conf` (server files; phase 8 lists the lines).

Proof: running it twice succeeds with "no changes"; `--check` passes; `psql -c '\du'` shows the three roles with the attributes above; `\l` shows `tenant_template` flagged as a template; the step-5 checks are also a Pest test against the local server.

1.2 Doctrine configuration

`config/packages/doctrine.yaml` : connections `landlord` and `tenant` (with `server_version: '17'`), `orm.auto_mapping: false` , entity managers `landlord` (mappings Tenancy, Iam) and `tenant` (business modules; empty for now). Disable `result_cache` and second-level cache on `tenant` . Two migration configurations: `migrations/landlord` for the landlord manager, `migrations/tenant` run by the custom command in 1.5.

Proof: `bin/console doctrine:mapping:info --em=landlord` lists only Tenancy and Iam entities; a Pest test asserts `getManagerForClass()` for every mapped class.

1.3 Landlord schema migration

One hand-written migration (DBAL, not generated) creating schemas `tenancy` , `iam` , `audit` , `messaging` and the 23 tables from section 4 of the v4 tab, with:

- CHECK constraints for every value set;
- composite unique keys `profile(id, tenant_id)` , `access_scope(id, tenant_id)` and the composite foreign keys that use them;
- partial unique indexes (`tenant` one platform row; `tenant_domain(host)` where active or failing; `tenant_job(tenant_id)` where running; `invitation(tenant_id, email)` where open);

- constraint triggers `profile_role_tenant_check`, `profile_scope_check`, `profile_service_account_check`, `role_permission_scope_check`, `api_key_service_account_check`;
- `audit.event` partitioned by month with a DEFAULT partition and the first six monthly partitions;
- hot-path indexes from section 4.6.

Add `schema_filter: ~^(?!audit\.)~` so Doctrine's diff ignores the partitioned table, and mark composite keys and triggers in a `SchemaListener` so `doctrine:schema:validate` stays clean.

Proof: `bin/console doctrine:migrations:migrate --em=landlord`; a Pest test queries `pg_constraint`, `pg_indexes` and `pg_trigger` and asserts each named constraint, index and trigger exists.

1.4 Entities and repositories (landlord)

Entities in `Tenancy/Infrastructure/Doctrine` and `Iam/Infrastructure/Doctrine` map single-id associations plus plain `tenantId` fields (Doctrine cannot join on non-primary keys). Enums as PHP backed enums with DBAL string type. Repositories for tenant-bound `iam` entities take a `TenantId` in every finder. A Doctrine SQL filter `tenant` on the landlord manager adds `tenant_id = :current` for those entities and is enabled by the tenant listener.

Proof: unit tests for each enum mapping; a test that the filter is enabled whenever a tenant is resolved.

1.5 Tenant template and `tenant:migrate`

`tenant:migrate` opens the direct `TENANT_MIGRATION_URL` with `dbname` replaced, runs the `migrations/tenant` set, and updates `tenant_database.schema_version` and `migrated_at`. Options: `--template` (migrate `tenant_template` only), `--pool` (all `ready` rows), `--tenant=<slug|id>`, `--all` (template, pool, then tenants by wave, creating `migrate` jobs with a shared `rollout_id`). The first tenant migration creates the business schemas (empty for now, with the migration history table).

```
bin/console tenant:migrate --template
```

Proof: `psql tenant_template -c '\dt *.*'` shows the history table; running `--template` twice is a no-op.

Phase 2: shared tenant kernel

All in `src/Shared/Tenancy`. No module code touches a tenant connection except through these.

2.1 Value objects and context

- `TenantId` (UUID v7 wrapper), `TenantRequirement` enum (`required`, `optional`, `none`, `platform`), `ResolvedTenant` (readonly: id, slug, type, status, verificationStatus, displayName, canonicalUrl, branding).
- `TenantContext`: final class, `set()`, `get()`, `has()`, `clear()`; throws `TenantAlreadySet` and `TenantNotResolved`; implements `ResetInterface`.
- Exceptions implementing `ApiPlatform\Metadata\Exception\ProblemExceptionInterface`: `TenantNotFound` (404, `type: tenant-not-found`), `TenantSuspended` (423), `ConflictingTenantHints` (400), `TenantNotReady` (503 with `Retry-After`), `TenantNotResolved` (500, logged as a bug).

Proof: unit tests for every exception's status and type; `TenantContext` double-set throws.

2.2 Ports

- `TenantDirectory` interface: `bySlug()`, `byHost()`, `byId()`, `canonicalUrl(TenantId)`. In-memory fake in `tests/Support`.
- `TenantConnectionResolver` interface: `forCurrentTenant(): ConnectionTarget` (pooledHost, pooledPort, databaseName, tenantId). In-memory fake.
- `CurrentActor` interface (Shared/Identity): `accountId()`, `profile()`, `session()`, `isPlatform()`.

2.3 `TenantResolver`

Pure class. Input: `Host` (normalised: lower case, no port, no trailing dot, punycode, length checks), optional header value, requirement. Steps: classify host (platform host from `tenancy.platform_hosts`; one label under `TENANCY_BASE_DOMAIN`; deeper → 404; else custom domain); header rules (must match on tenant-bound hosts; required on shared hosts for `required` routes; slug, id or retired slug accepted); directory lookup; status outcome. Returns `ResolvedTenant` or null for `none` / `optional` without a tenant.

Proof: table-driven unit tests covering every row of section 8 of the v4 tab (conflict 400, deeper subdomain 404, unknown host 404, suspended 423, provisioning 404, retired slug accepted, `X-Tenant` on a portal host must match).

2.4 `TenantScopedRunner`

`run(TenantId, callable)`: refuse nesting; refuse if the tenant connection has an active transaction; look up the tenant through `TenantDirectory::byId`; set context; run; `finally`: `rollBack()` if active, `clear()` both managers, `close()` the tenant connection, `resetManager('tenant')` if closed, restore previous context.

Proof: integration test with two tenant databases: run A then B, B sees no entity from A, `pg_stat_activity` shows a new connection for B.

2.5 Adapters

- HTTP: `ResolveTenantListener` on `kernel.request` priority 20 (reads `_tenant` default, calls the resolver, sets context and `_tenant_id` attribute, enables the Doctrine `tenant` filter). `CloseTenantConnectionListener` on `kernel.terminate` (roll back, close, clear context).
- Messenger: `TenantStamp`; `TenantMiddleware` (on dispatch: add stamp from context or throw `TenantMessageWithoutContext` for `TenantScopedMessage`; on receive: run the handler in the runner; stamp missing → `UnrecoverableMessageHandlingException`). `LandlordMessage` marker interface. Middleware order: `tenant` before `doctrine_ping_connection` and `doctrine_transaction` for the tenant connection. `WorkerMessageHandled` / `WorkerMessageFailed` listener closes the connection.
- Console: `TenantScopedCommand` trait adding `--tenant` and `--all`, running the body in the runner once per tenant.

Proof: functional tests: request without `_tenant` on a route fails the architecture test; request to `a.localhost` then `b.localhost` on one kernel leaks nothing; message without stamp lands in the failure transport; command `--all` visits every active tenant.

2.6 Route requirement compilation

- API Platform: `TenantRequirementMetadataFactory` decorating `api_platform.metadata.resource.metadata_collection_factory`: reads `extraProperties['tenant']` (default from

`api_platform.defaults.extra_properties.tenant: required`) and writes `defaults['_tenant']`; throws during cache warm-up if an operation has neither.

- Controllers: `TenantRequirementRouteLoader` extending `AttributeRouteControllerLoader::configureRoute()` reading `#[TenantRequirement]`; registered as `routing.loader.attribute`.
- Architecture test iterates `RouterInterface::getRouteCollection()` and fails on any route without `_tenant`.

2.7 Connection middleware

`TenantConnectionMiddleware` (`#[AsMiddleware(connections: ['tenant'], priority: 100)]`) wrapping the driver in `TenantDriver`: on `connect()`, ask the resolver, replace `host`, `port`, `dbname`, add `application_name = tenant:<id>`, connect, run `SELECT current_database()` and compare, return `TenantBoundConnection` (checks `TenantContext` before `prepare`, `query`, `exec`, `beginTransaction`).

Proof: integration tests: query with no context → `TenantNotResolved` and no connection opened; context swapped without the runner → `TenantConnectionMismatch`; `application_name` visible in `pg_stat_activity`.

Phase 3: Tenancy module

3.1 Domain

Aggregates: `Tenant` (status transitions, slug rules, verification), `TenantDomain` (verification lifecycle), `TenantDatabase` (pool states), `TenantJob`. Domain events: `TenantCreatedV1`, `TenantActivatedV1`, `TenantSuspendedV1`, `TenantReactivatedV1`, `TenantArchivedV1`, `TenantVerifiedV1`, `TenantSlugChangedV1`, `TenantDomainActivatedV1`, `TenantDomainDeactivatedV1`, `TenantDatabaseAssignedV1`, `TenantProvisioningFailedV1`, `TenantReadyV1`. Slug validator with the reserved list in `config/tenancy/reserved_slugs.php`.

Proof: unit tests for every transition and every slug rule.

3.2 Configuration

`config/packages/tenancy.yaml` with `base_domain`, `platform_hosts`, `clusters.<code>.{pooled, direct, template_database, pool.min_ready, capacity.max_databases, accepting_new}`. A `ClusterRegistry` service reads it;

`ClusterSelector` picks the accepting cluster with the fewest assigned databases.

3.3 `TenantDirectory` implementation

Doctrine queries on `tenant`, `tenant_domain`, `tenant_slug_history`; Redis cache 60 s under `tenancy:slug:*`, `tenancy:host:*`, `tenancy:id:*` tagged `tenant_<id>`; misses cached for classified names only; single-flight via `symfony/lock`; invalidation listener on every Tenancy event and on domain changes.

Proof: cache hit and miss tests; invalidation after `TenantSlugChangedV1`; `canonicalUrl` prefers the primary active domain.

3.4 `TenantConnectionResolver` implementation

Reads `tenant_database` by tenant id (in-process cache 10 min, invalidated on `TenantDatabaseAssignedV1`), joins `cluster_code` to `ClusterRegistry`, returns the pooled endpoint and database name.

3.5 Pool filler

- `TenancySchedule` (`#[AsSchedule('tenancy')]`): `CheckTenantDatabasePool` every minute, stateful and locked.
- `CheckTenantDatabasePoolHandler`: per cluster, count `ready`; dispatch `FillTenantDatabasePool(cluster)` per missing row, capped by `capacity.max_databases`.
- `FillTenantDatabasePoolHandler` (`LandlordMessage`): insert `creating` row; on the cluster's direct endpoint as the migrator: `CREATE DATABASE t_<id> TEMPLATE <template> STRATEGY = WAL_LOG` (autocommit); connect to it; grants and `ALTER ROLE colsynq_app IN DATABASE … SET …`; read migration version; update row to `ready`; `PoolerConfigWriter::add()` (no-op locally, PgBouncer entry plus reload on servers).
- Reaper: `creating` rows older than 5 minutes are dropped and deleted.
- Messenger retry for the `tenant_provisioning` transport: `max_retries: 5, delay: 2000, multiplier: 2, max_delay: 60000`, failure transport `failed`.

```
bin/console messenger:consume scheduler_tenancy tenant_provisioning -vv
psql colsynq_landlord -c "select database_name, status from
tenancy.tenant_database"
```

Proof: after one tick, `min_ready` databases exist and are `ready`; killing the worker mid-fill leaves a `creating` row that the reaper removes; the template with an open connection makes the step retry, not fail.

3.6 `CreateTenant` and the onboarding saga

- Command `CreateTenant(slug, names, registration, origin, ownerEmail, idempotencyKey)`; handler in one landlord transaction: slug validation, `tenant` insert, pool claim (`FOR UPDATE SKIP LOCKED`), `tenant_job` insert with `state = database_claimed`, outbox `TenantCreatedV1`. Empty pool: synchronous create with a 5 s budget via the filler's steps, else `state = requested` and the asynchronous path.
- `config/packages/workflow.yaml`: state machine `tenant_onboarding` on `TenantJob::state`, places and transitions from section 5.4 of the v4 tab; a `GuardListener` per transition checks the precondition.
- One message class per transition (`VerifySchema`, `Seed`, `PrepareStorage`, `Activate`, `MarkReady`), each `LandlordMessage`, each dispatched by the previous handler with `DispatchAfterCurrentBusStamp`; heartbeat update in each handler; the reaper fails jobs with a stale heartbeat.
- Compensation: on terminal failure, release the database to the pool if only seed data exists (`seed_version` present, no other rows), else `retired`; `TenantProvisioningFailedV1`.
- `POST /platform/tenant-jobs/{id}/retry` re-dispatches the message for the current state.

Proof: functional test creates a tenant and asserts `active` within 2 s locally; chaos test kills the worker after each transition and resumes; two concurrent `CreateTenant` calls claim two different databases; idempotency key returns the same tenant.

3.7 Lifecycle commands and platform API

API Platform resources (DTOs in `Tenancy/Infrastructure/Api`) with `extraProperties: ['tenant' => 'platform']`: `POST /platform/tenants`, `GET /platform/tenants`, `/{id}`, `POST .../suspend`, `/reactivate`, `/archive`, `/verify`, `/slug`, `/owner-invitation`, `GET/POST /platform/tenants/{id}/domains`, `GET /platform/tenant-jobs`, `POST .../retry`, `POST /platform/rollouts`. Each processor dispatches a command; security `is_granted('perm:tenants:manage')`.

Archive: domains to `inactive`, `ALTER ROLE colsynq_app IN DATABASE t_<id> SET default_transaction_read_only = on`, `TenantArchivedV1`.

Proof: API tests for each endpoint as a platform admin and as a society admin (403).

3.8 Custom domains

- `AddDomain`: PSL check (`php-domain-parser`), not under base domain, insert `pending` with a 128-bit token.
- `VerifyDomain` (manual `POST .../domains/{id}/verify` and daily scheduled check): DNS TXT `_colsynq-challenge.<host>` or HTTP file; routing check (CNAME to `domains.<base>` or A/AAAA to configured addresses); both pass → `active` + `TenantDomainActivatedV1`; failures → `failing`, after 7 days `inactive`.
- `CertificateIssuer` port: local no-op; production implementation runs Certbot webroot, writes the Nginx server block from a template, reloads Nginx (Phase 8).

Proof: tests with a fake DNS resolver for each outcome; activation deletes the directory's miss entry.

Phase 4: IAM core (accounts, profiles, roles)

4.1 Catalogue in code

`PermissionProvider` (every `module:resource:action` with risk level and allowed scope types), `SystemRoleCatalogue` (`platform_admin`, `society_admin`, `society_staff`, `self_service`, with ranks), `iam:sync-catalogue` command (upsert `permission` and system roles, mark removed permissions deprecated, fail if a custom role uses a deprecated one or any permission is marked financial).

Proof: command is idempotent; a custom role on a deprecated permission fails the command.

4.2 Tenant policy from events

Inbox consumers (`messaging.inbox` keyed by consumer + event id): `TenantCreatedV1` → `tenant_policy` (`provisioning`, `closed`, default role `self_service`, `source_version`), plus an unsent bootstrap `invitation` for console-created tenants; `TenantActivatedV1` → active, send the invitation, `InvitationSentV1`; status events applied only when `source_version` is newer; unit and venture events → `access_scope`.

Proof: delivering the same event twice changes nothing; an older version after a newer one is ignored.

4.3 Accounts and profiles

- `RegisterUserInTenant` handler: email proven first (signed link), `INSERT … ON CONFLICT` on email and on (`user_id`, `tenant_id`), join-mode rules, role from `default_role_id`, events `UserRegisteredV1`, `ProfileCreatedV1`, `UserJoinedAdditionalTenantV1` when the account already had a profile elsewhere.
- Existing email at sign-up: same generic response, "sign in to join" e-mail.
- Profile commands: change role or scope (`profiles:assign-role`, rank check, not self), disable, remove, leave, with the last-administrator guard (`SELECT … FOR UPDATE` on `tenant_policy`, then count), `access_version` bump in the same transaction.
- Invitations: create (rank check, one open per tenant and email, revoke the expired one), accept (email must match, role still valid, scope still open), revoke.
- Account deactivation: lock tenants in id order, revoke sessions, disable profiles.

Proof: concurrency test with two administrators removing each other; sign-up double submit; join `closed` without invitation refused; `left` can rejoin, `removed` cannot.

4.4 Society and `/me` APIs

Society administration (`tenant` requirement): `/profiles` (list, patch role, disable, remove), `/invitations`, `/roles` (custom roles with permissions, within the catalogue), `/service-accounts`. `/me` (`none` requirement): account, `is_multi_tenant`, `profiles[]`, `POST /me/profiles` (join), `DELETE /me/profiles/{id}` (leave), sessions, authenticators, external identities.

Proof: API tests as `society_admin`, `society_staff`, `self_service`, and as a profile of another tenant (403 everywhere).

4.5 Authorisation

`PermissionVoter` for `perm:*` attributes, `AccessEvaluator` (role permissions cached under profile, `access_version`, catalogue version), assertions tagged `iam.assertion` with `#[AutowireLocator]: HasPermission`, `ProfileIsActive`, `TenantIsActive`, `TenantIsVerified`, `RecentAuthentication`, `ResourceInScope`.

`security.access_decision_manager.strategy: unanimous`. `ScopeExtension` for API Platform collections and items. Limited-mode rules for unverified tenants (invitation cap, no live service clients) as assertions.

Proof: voter unit tests per assertion; `Challenge` becomes 401 with the RFC 9470 header; collection filtered by scope.

Phase 5: authentication

5.1 Authorization server at `auth.`

- `league/oauth2-server-bundle`: grants `authorization_code` (PKCE S256 required for public clients), `client_credentials`, `refresh_token` (rotation on, families tracked in `refresh_token.family_id`); `implicit` and `password` disabled; PAR endpoint; token lifetime 10 min; refresh lifetimes per client type.
- First-party clients registered by `iam:bootstrap-platform` from `config/iam/clients.php`: `web-bff` (confidential), `mobile` (public, `require_dpop`), `console-bff` (confidential). Society confidential clients created by `/service-accounts`.
- Login UI on `auth.` (stateful firewall `auth`): password form, passkey (webauthn bundle), TOTP, backup codes, society picker when the account has several profiles and the client sent no `tid`, MFA enforcement by the target tenant's `mfa_policy`.
- Custom grant `urn:ietf:params:oauth:grant-type:token-exchange` for society switching (`requested_tid`), enforcing the target's `mfa_policy`.
- Access token issuer: `web-token` JWS ES256, `kid` from `signing_key`, claims from section 11.2 of the v4 tab; `/.well-known/jwks.json`; `iam:rotate-signing-key` command (next → current → retired).
- Session and token tables: `auth_session`, `refresh_token`; revocation endpoint writes `jti`, `sid` to Redis.

Proof: conformance tests: PKCE missing → error; code reuse → both tokens revoked; refresh reuse → family revoked; `auth_time` unchanged after refresh; JWKS contains current and next keys.

5.2 Resource server

Symfony `access_token` authenticator on the `api` firewall with `ColsynqAccessTokenHandler`: parse JWS, pick key by `kid`, verify `alg`, `typ`, `iss`, `aud`, `exp`, `jti` not revoked, `ver` equals `token_version`, `tid` equals the resolved tenant (unless `none` / `platform` routes), DPoP proof when `cnf.jkt` present (nonce and `jti` replay cache in Redis). Builds the `CurrentActor`. API-key authenticator for `Authorization: ApiKey` (HMAC with pepper, scopes, IP allow-list). Redis unavailable → database check, never allow.

Proof: tests for each rejected claim; `tid` mismatch 403; DPoP replay refused; Redis down still rejects a revoked `sid`.

5.3 Web and mobile clients

- BFF for the web app (same origin as the SPA): `__Host-` session cookie, CSRF token, `Sec-Fetch-Site` / `Origin` check, server-side token store in Redis, single-flight refresh.
- Portals on custom domains: `redirect_uri` registered on domain activation (`https://<host>/auth/callback`).
- Mobile: documented flow (PAR → system browser → code → token with PKCE and DPoP).

Proof: BFF functional test: sign-in, call API, refresh, sign-out everywhere ends the second session.

5.4 Third-party sign-in

Relying party at `auth.`: `/auth/oidc/{issuer}/start` and `/auth/oidc/callback` with `state`, `nonce`, PKCE; ID token validation with the issuer's JWKS; match on (`issuer`, `subject`); unknown identity with an existing email → "sign in to link"; never auto-link.

5.5 Throttling and lockout

Redis rate limiters per (IP, account), per IP, per account, per tenant, applied before password hashing; soft lock with passkey or e-mail unlock.

Phase 6: audit and messaging infrastructure

- `AuditWriter` (Shared): inserts into `audit.event` with DBAL inside the current landlord transaction; every command handler in Tenancy and IAM calls it; security outcomes (failed sign-in, denied, cross-tenant attempt) written by the authenticators and voters.
- `audit:create-partitions` scheduled monthly, creating three months ahead; alert if the DEFAULT partition receives rows.
- Outbox relay (`messaging:relay-outbox`, scheduled every 5 s, locked): publishes unpublished rows to the `events` transport; inbox middleware for consumers.
- Log processor masking e-mails, tokens and secrets.

Proof: each command leaves exactly one audit row; relay publishes once; consumer redelivery is a no-op.

Phase 7: API Platform conventions and platform console

- `api_platform.yaml`: `defaults.stateless: true`, `defaults.extra_properties.tenant: required`, `exception_to_status` copied in full plus the tenant exceptions, `swagger.api_keys` bearer, no GraphQL.
- `OpenApiFactory` decorator adding the `X-Tenant` header parameter to shared-host operations and `servers` for `https://{tenant}.<base>`.
- Problem responses: test that a `kernel.request` exception renders `application/problem+json`.
- Console (`console.` host, `platform` requirement): tenant list and detail, jobs, domains, rollouts, bootstrap status.

Phase 8: production on a VPS (Nginx, PHP-FPM, PgBouncer)

1. Packages: `php8.4-fpm`, `postgresql-17`, `pgbouncer`, `redis-server`, `nginx`, `certbot` with the DNS plugin for the registrar.
2. PostgreSQL: `listen_addresses = 'localhost'`, `pg_hba.conf` `hostssl all colsynq_app,colsynq_migrator 127.0.0.1/32 scram-sha-256`; run `tenancy:setup-cluster vps-1 --admin-url=…`.

3. PgBouncer: transaction mode, `auth_query` against `pgbouncer_auth` (created by `setup-cluster`), `[databases]` file generated by `PoolerConfigWriter` and included; `systemctl reload pgbouncer` after each pool fill. Clusters config: `pooled: 127.0.0.1:6432`, `direct: 127.0.0.1:5432`.
4. Nginx: wildcard server block for `<base>` and `*.<base>` with the DNS-01 certificate; default server returning 444; per-domain blocks written by the certificate issuer to `sites-enabled/tenants/`.
5. PHP-FPM pool sizing; `opcache.validate_timestamps = 0`; deploy script runs migrations, `iam:sync-catalogue`, `cache:warmup`, `messenger:stop-workers`.
6. systemd units: `messenger-worker@async`, `messenger-worker@tenant_provisioning`, `messenger-worker@events`, `scheduler` (`messenger:consume scheduler_tenancy scheduler_iam`).
7. Secrets: OpenBao or `0600` env files; database passwords and signing keys never in the repository.
8. Backups: nightly base backup plus WAL archiving to object storage in Nigeria; a documented single-society restore.
9. Monitoring: pool size per cluster, job failures, certificate expiry, audit DEFAULT partition, PgBouncer pool saturation.

Proof: go-live checklist from section 16.3 of the v4 tab, each item ticked by the person who verified it.

Phase 9: test suites and definition of done

Suite	Runs	Covers
`tests/Architecture`	every commit	module dependencies, `_tenant` on every route, `security` on every write operation, entity-manager map, messages are tenant or landlord
`tests/Catalogue`	every commit	every hand-written constraint, index and trigger exists
`tests/Isolation`	every commit, two tenant databases	no context fails closed; mismatch throws; `current_database()`; A then B leaks nothing; worker stamp paths

Suite	Runs	Covers
`tests/Tenancy`	every commit	resolver table, pool filler, saga resume, domains
`tests/Iam`	every commit	registration, profiles, invitations, voters, OAuth conformance, token handler
Chaos (manual)	before release	kill worker per transition; PgBouncer restart; Redis down

Definition of done for the whole plan: a new society can sign up on the Mac in under 2 seconds, its administrator signs in through the authorization server, calls a tenant API that hits its own database, and every suite above is green.

Suggested order and sizing

Phase	Depends on	Rough effort
0 Workstation	—	half a day
1 Database foundation	0	3 days
2 Shared kernel	1	4 days
3 Tenancy	2	6 days
4 IAM core	2, 3.1–3.4	6 days
5 Authentication	4	8 days
6 Audit and messaging	2 (can start with 3)	2 days
7 API Platform and console	3, 4	3 days
8 VPS	1–7	3 days
9 Suites	alongside each phase	continuous

Phases 3 and 4 can run in parallel after 2 and 3.1–3.4; phase 6 can run alongside 3.